Look-See-Do Fighter

AI을 통한 격투 게임 리팩토링 및 개발 자동화

개발을 중단했던 2D 온라인 격투 게임을 다시 분석하고 기존 전투 콘텐츠를 보존하면서 게임플레이를 개선했습니다.

Codex와 Unity MCP를 활용해 코드 조사부터 기능 구현, 테스트 및 검증 결과 관리까지 이어지는 AI Native 개발 워크플로를 구축했습니다.

결정론적 네트워크 · 기존 시스템 리팩토링 · AI 개발 워크플로 · 자동 검증

TECH STACK

Game Engine	Unity 6 / C#
Networking	Photon Quantum 3
Architecture	Deterministic ECS
AI Development	Codex CLI(Astra,Sol) / Unity MCP
Automation	AGENTS.md · Skills · Hooks · Git/GitHub
Testing	Unity Test Framework / NUnit / Python

Look-See-Do Fighter란?

사용할 기술을 직접 구성하여 자신만의 운영으로 1대1 대전을 하는 게임입니다.

개발 인원 : 프로그래머 1명(본인), 애니메이터 1명
개발 기간 : 6주

기술 커스터마이징으로 나만의 전략 수립

기술별 비용과 예산 제한 안에서 기술들을 전략적으로 선택
왼손/왼발/오른손/오른발 + 방향키의 직관적인 조합으로 전투

프레임 기반 전투

이하의 요소들을 통해 전투를 진행

- 기술별 발동 프레임/후딜레이 프레임/가드 시 경직 프레임
- 상단/중단/하단의 판정
- 패링 자세와 패링 불가 기술로 패링을 파훼

Quantum 3 를 통한 쾌적한 온라인 환경

Quantum 3의 결정론적 환경을 통해 판정이 엇갈리지 않음
10~12ms 사이를 유지하는 핑

GAMEPLAY CAPTURE



게임 플레이 영상
https://youtu.be/C8R3RhYLBAw

프로젝트 재개와 개발 목표

컨텐츠는 유지하고

구조는 다시 정리

해당 프로젝트는 이미 기본적인 전투·공격 기술·애니메이션·네트워크 구조가 어느정도 구현되어 있던 프로젝트였습니다.

따라서 에셋과 로컬 데이터에 의존한 실행 상태, 공격별 중복 코드, 복잡한 상태 등의 유지보수가 부담으로 이어졌습니다.

개발 목표

01 AI 개발 체계

Skills·Hooks·MCP·테스트·버전 관리 표준화

02 리팩토링

Quantum Simulation과 Unity View 책임 분리(ECS 구조)

03 기존 콘텐츠 보존

37종 공격 타격 시점·애니메이션 참조 유지 및 변경

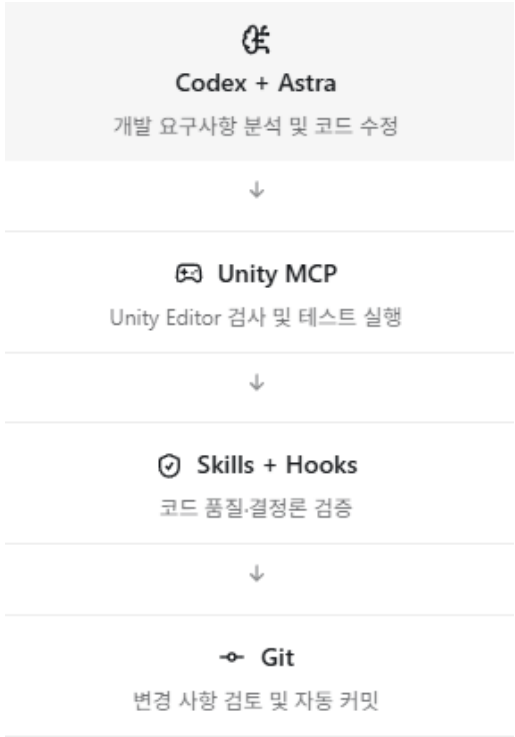
04 게임플레이 확장

선입력·3선승·공중 콤보·전투 연출

AI 개발 환경

AI가 프로젝트 제약을 이해하고 검증 가능한 결과를 만들도록 작업 지식을 파일로 유지했습니다.

구성 요소	프로젝트에서의 활용
Codex CLI	Astra : 코드 분석 및 개발 환경 세팅 Sol : 코드 구현 및 리팩토링
Unity MCP	Editor 조사 및 테스트 실행
AGENTS.md	프로젝트 규칙 및 기술 제약 관리
Skills	작업별 절차와 검증 기준 표준화
SessionStart Hook	세션 재개 시 규칙과 Tool 사용 안내
Python Guard	Simulation 위험 패턴을 정적으로 검사
Git / GitHub	변경 관리·검토·게시 절차



프로젝트 전용 Skills로 AI 개발 절차 표준화

4개 Skills로 개발 작업과 검증 기준을 표준화했습니다.

Skill	역할	핵심 내용
lsdf-game-development	기능·버그·UI·밸런스	전투 기능, UI, 밸런스·연출 개선의 구현 절차 정의
lsdf-quantum-refactor	상태 소유권·구조 리팩터링	Frame 상태 소유권, Simulation/View 분류, 기존 타이밍·에셋 참조 보존
lsdf-quantum-verify	CodeGen*·컴파일·재생 검증	변경 범위에 필요한 CodeGen·컴파일·테스트·재생/롤백 검사 선택
publish-unity-pr	Git 변경 검토·게시 준비	변경 파일·검증 근거·비밀정보 검토 및 사용자 승인 후 Push와 PR 진행

CodeGen* : qtn이라는 Photon사의 언어로 쓰인 코드를 C# 코드로 생성하는 것

개발 및 검증 워크 플로우

요구사항 → 조사 → 구현 → CodeGen/컴파일 → 정적 검사 → Unity 테스트 → 기록 → Git 게시

Unity-MCP를 이용한 테스트 결과

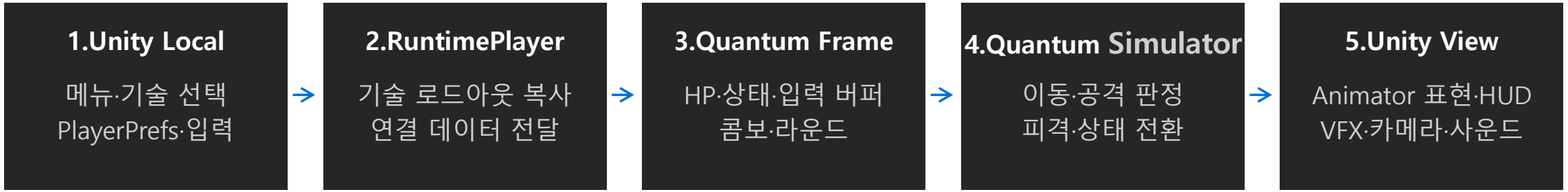
테스트 내용	테스트 결과
선입력·매치	157 PASS
로드아웃·체력 연출	235 PASS
자연 재타격 낙하	106 PASS
공중 피격 밀림	99 PASS
Sprite 관련 검증	66 PASS
배경·Photon 네트워크 연결	30 PASS

GitHub 게시 후보 검토

- 기존 저장소의 브랜치·변경 확인
- 관련 파일만 staging, 비밀·대형 파일·이력 검사
- 테스트 후보와 커밋 대상 tree 일치 확인
- publish-unity-pr · 승인 범위의 Commit/Push

이러한 환경을 구축 한 뒤 개발에 들어갔습니다.

리팩토링 : Simulation과 Unity View의 책임 분리



상태와 데이터

1. Unity 엔진을 통해 기술을 선택

2. RuntimePlayer로 선택한 기술의 로드 아웃을 전달

3. Quantum Frame에 실시간 상태를 저장하여 Quantum Simulator로 전달

판정과 표현

•4. Quantum Simulator를 통한 결정론적 판정

플레이어들의 커멘드 입력만 Quantum 서버의 시뮬레이터로 전송하고 시뮬레이터 안에서 판정을 정하고 그 결과를 플레이어들에게 전달하는 결정론적 방식

•4. Unity View를 통해 Simulator 결과를 표현

37종의 기술들의 데이터 정리

타격 데이터와 판정 콜라이더 생성 절차를 분리하고, 공격별 고유 동작은 유지했습니다.

PROBLEM

피해·경직·피격 높이·히트박스 생성 코드가 기술마다 중복되어 이로 인해 유지 보수와 변경이 어려웠습니다

<기술마다 직접 히트 박스를 생성>

```
[Serializable]
public class JapJapWindowEvent : AnimatorTimeWindowEventAsset
{

    f.Add(hitbox, new PhysicsCollider2D
    {

        IsTrigger = true,
        //박스 크기
        Shape = Shape2D.CreateBox(new FPVector2(FP._0_20 / 2, (FP._0_10 - FP._0_02) / 2))
    });
```

SOLUTION

역할을 기준으로 나눠 리팩토링하여 생산성을 높혔습니다.

구성 요소	담당 역할
LSDFSingleHitDefinition	단타 피해·경직·판정 데이터
LSDFTimedHitDefinition	다단 생성 프레임·타격 데이터
LSDFHitboxFactory	Collider·Hitbox 생성/파괴 시점
기존 Window Event	이동·연계·특수 동작·종료

<히트박스 생성은 LSDFHitboxFactory에서 일괄적으로 관리 >

```
namespace Quantum.LSDF {
  /// <summary>Stateless single-hit materialization. Multi-hit scheduling is intentionally separate.</summary>
  public static class LSDFHitboxFactory {
    public static EntityRef Spawn(Frame frame, EntityRef attacker, int facing, LSDFSingleHitDefinition hit,
        FPVector2? effectPositionOverride = null, AssetGuid sourceAttack = default) {
      EntityRef entity = frame.Create();
      ConfigureBody(frame, entity, attacker, facing, hit);
      AddPayload(frame, entity, attacker, hit, effectPositionOverride, sourceAttack);
      ScheduleDestroy(frame, entity);
      return entity;
    }
```

프레임 단위 선입력 시스템 구현

A기술 재생 중
B 기술 커맨드 입력

→

대기

가장 마지막에 입력한
커맨드를 전달

→

전달 중

A 기술의 재생되는 동안
커맨드 유지

→

실행

A 기술이 끝난 후
바로 B 기술 재생

다음 기술을 미리 입력해도 입력이 사라지지 않도록 대기와 전달 상태를 나눴습니다.

재생되고 있는 기술이 끝나기 20프레임 전에 눌렀던
커맨드 중 가장 최근에 누른 커맨드를 실행하도록 구현하여

상대의 기술을 가드하고 프레임 손실 없이 바로 반격 할 수 있는 시스템을 마련했습니다

게임 루프

로비 → 매칭 시작 → 대전 시작 → 5판 3선 끝 → 로비

주요 구현

- KO 결과를 한 번만 확정해 중복 처리 방지
- 동시 KO는 무득점, 같은 라운드 재시작
- HP·위치·공격·버퍼·콤보 상태 초기화
- 승수·기술 로드아웃은 라운드 사이 유지
- 매치 종료 → 연결 종료·메인 복귀

INGAME CAPTURE



전투 연출

Quantum 이벤트와 Unity 연출 연결

- Simulation에서 일반 타격·가드·카운터·패링 이벤트 발생
- Unity View가 이벤트를 받아 해당 위치에 프리팹 재생
- 전투 판정과 시각적 표현의 책임 분

예측 이벤트와 이펙트 수명 관리

- 이벤트 키를 이용해 동일 효과의 중복 생성 억제
- 예측 이벤트가 취소되면 연결된 이펙트 제거

체력과 카메라 연출

- HP 변화에 따른 체력바 흔들림
- 카운터 시 카메라 줌 설정
- Follow → Zoom → Shake 변환 합성

INGAME CAPTURE





프로젝트 성과 및 개선 과제

문제를 정의하고 기술적 제약을 전달하며 AI의 결과를 검증할 수 있는 판단력을 쌓았습니다.

성과

01 **ECS 시스템 설계**

02 **게임플레이 문제 해결 및 개선**

03 **AI 개발 환경 구축**

AGENTS·Skills·Hook·MCP·Guard와 인계

04 **검증·변경 이력 관리**

실행 근거·한계 기록, Git 후보·상태 추적

남은 개선 과제

- 다양한 상황에서의 네트워크 품질 확인
- 다양한 기기에 배포 검증
- 매칭이 안될 때를 대비한 AI npc